



UNIVERSIDAD DE LAS CIENCIAS INFORMÁTICAS
FACULTAD DE TECNOLOGÍAS INTERACTIVAS

DESARROLLO DE UN GENERADOR DE CONTENIDO PARA JUEGOS CON RECURSOS LIMITADOS

Trabajo de diploma para optar por el título de Ingeniero en Ciencias Informáticas

Autor: Antonio Luis Farrés Corzo

Tutor: Omar Correa

La Habana, Marzo de 2026

Año del Centenario del Comandante en Jefe Fidel Castro Ruz

Escribe aquí tu pensamiento o frase inspiradora
Autor del pensamiento

Dedicatoria

Escribe aquí tu dedicatoria

Agradecimientos

Escribe aquí tus agradecimientos

Declaración de autoría

Declaramos ser autores de la presente tesis y reconocemos a la Universidad de las Ciencias Informáticas los derechos patrimoniales sobre esta, con carácter exclusivo.

Para que así conste firmamos la presente a los ____ días del mes de _____ del año _____.

Antonio Luis Farrés Corzo
Autor

Omar Correa
Tutor

La investigación aborda la ineficiencia en la gestión de activos digitales en equipos creativos que operan con recursos tecnológicos limitados. El objetivo general consistió en desarrollar y validar una aplicación web, denominada Assets Studio, orientada a organizar, recuperar y reutilizar recursos multimedia de forma eficiente en el contexto cubano. El diseño metodológico integró Design Thinking para la identificación de necesidades y el prototipado centrado en el usuario, junto con Scrum como marco de desarrollo iterativo para la implementación técnica. El estudio incluyó entrevistas semiestructuradas, modelado de requisitos, construcción incremental del producto y pruebas de usabilidad con usuarios del sector creativo. Como resultado, se implementó una solución con autenticación, carga y categorización de recursos, búsqueda semántica, generación asistida por inteligencia artificial e interacción colaborativa. La validación evidenció una disminución del tiempo de localización de activos y una mejora en la organización del flujo de trabajo. Se concluye que la propuesta es técnicamente viable y metodológicamente pertinente para contribuir a la informatización de procesos creativos en escenarios de conectividad y recursos restringidos.

Palabras clave: Gestión de activos digitales, desarrollo de software, Design Thinking, Scrum, búsqueda semántica.

Lista de tareas pendientes	xiii
Introducción	1
1 FUNDAMENTOS Y REFERENTES TEÓRICO-METODOLÓGICOS SOBRE EL OBJETO DE ESTUDIO	2
1.1 Referentes teórico-metodológicos sobre el objeto de estudio	2
1.2 Fundamentos teórico-metodológicos del desarrollo tecnológico	3
1.3 Diagnóstico del objeto de estudio: aplicación para gestión de activos digitales	4
1.4 Fundamentos teórico-metodológicos de las tecnologías y herramientas utilizadas	5
2 DISEÑO DE LA SOLUCIÓN PROPUESTA AL PROBLEMA CIENTÍFICO	7
2.1 Epígrafe 1	8
2.1.1 Modelado de los procesos de gestión y generación de activos digitales	8
2.1.2 Identificación de Actores del Sistema	8
2.1.3 Modelado de Casos de Uso del Negocio	9
2.1.4 Subproceso de Publicación y Gestión de Assets	9
2.1.5 Subproceso de Generación de Contenido Asistido por IA	10
2.1.6 Subproceso de Interacción y Notificaciones	11
2.2 Epígrafe 2	11
2.2.1 Ingeniería de Requisitos, Análisis y Diseño de la Solución	11
2.2.2 Requisitos Funcionales del Sistema (RF)	11
2.2.3 Requisitos No Funcionales (RNF)	12
2.2.4 Arquitectura del Sistema	12
2.2.5 Diseño del Modelo de Datos	13
2.3 Epígrafe 3	14
2.3.1 Diseño de mecanismos de datos, implementación e interfaces de usuario	14
2.3.2 Mecanismos de Almacenamiento y Persistencia	15
2.3.3 Mecanismos de Procesamiento y Transmisión	15
2.3.4 Diseño de Interfaces Gráficas de Usuario (GUI)	15

2.4	Epígrafe 4	17
2.4.1	Diseño de la gestión de errores y despliegue de la solución	17
2.4.2	Estrategia de Gestión y Tratamiento de Errores	17
2.4.3	Diseño del Despliegue (Deployment)	18
2.4.4	Gestión de Variables de Entorno y Seguridad	20
2.5	Conclusiones del Capítulo	20
3	VALIDACIÓN DE LA SOLUCIÓN PROPUESTA	21
3.1	Estrategia de Verificación y Validación de la Solución	21
3.1.1	Pruebas de Funcionalidad y de Integración	21
3.1.2	Pruebas de Usabilidad	22
3.2	Impacto de la Solución en la Gestión de Activos Digitales	22
3.2.1	Análisis del Estado Actual vs. Estado Deseado	22
3.3	Estudio de Factibilidad Económica y Tecnológica	22
3.3.1	Factibilidad Tecnológica	23
3.3.2	Factibilidad Económica	23
	Conclusiones del capítulo	23
	Conclusiones	24
	CONCLUSIONES FINALES	25
	Recomendaciones	26
	RECOMENDACIONES	27
	Referencias bibliográficas	28
	Apéndices	29
A	Proyectos y Avaes	30
B	Otro reporte	31
B.1	Otra sección de muestra	31
B.2	Otro ensamble Industrial	32

Índice de figuras

2.1	Diagrama de Casos de Uso General del Sistema	9
2.2	Diagrama de Actividad del Proceso de Subida de Assets	10
2.3	Diagrama de Casos de Uso General del Sistema	13
2.4	Diagrama Entidad-Relación (DER)	14
2.5	Interfaz de la Página Principal y Galería de Assets	16
2.6	Interfaz de Detalle de Asset con Comentarios y Opciones	16
2.7	DInterfaz de Generación de Assets mediante IA	17
2.8	Diagrama de Casos de Uso General del Sistema	18
2.9	Diagrama de Casos de Uso General del Sistema	19

Índice de tablas

Lista de algoritmos

Lista de códigos fuentes

Lista de tareas pendientes

El panorama contemporáneo de las industrias creativas se caracteriza por una alta dependencia de activos digitales. Desde el diseño gráfico y la producción audiovisual hasta el desarrollo de videojuegos, la capacidad de crear, almacenar, localizar y reutilizar recursos digitales de forma eficiente constituye un factor crítico de productividad y competitividad. Sin embargo, la práctica profesional evidencia una gestión fragmentada de estos recursos, con archivos dispersos en múltiples soportes y servicios no integrados. Esta situación provoca pérdida de tiempo, duplicidad de versiones, dificultades de colaboración y disminución del rendimiento de los equipos creativos.

En este contexto, la investigación se orienta al desarrollo de Assets Studio como solución tecnológica para la gestión inteligente de activos digitales. El objeto de estudio se centra en los procesos de organización, recuperación y aprovechamiento de recursos multimedia en entornos con restricciones de conectividad e infraestructura. El campo de acción se delimita al diseño e implementación de una aplicación web con funcionalidades de clasificación, búsqueda semántica, generación asistida por inteligencia artificial e interacción colaborativa.

El diseño metodológico se estructura en dos componentes complementarios. En primer lugar, se aplica Design Thinking para comprender las necesidades de los usuarios, definir el problema y construir prototipos progresivos validados mediante pruebas de usabilidad (Brown, 2008; Nielsen, 1994). En segundo lugar, se adopta Scrum para planificar e implementar iteraciones de desarrollo con incremento funcional verificable en cada ciclo (Schwaber y Sutherland, 2020). La combinación de ambos enfoques permite articular una ruta de trabajo centrada en el usuario y, al mismo tiempo, técnicamente controlada.

La investigación emplea métodos teóricos y empíricos. Entre ellos se incluyen análisis documental, entrevistas semiestructuradas, modelado de requisitos, observación de flujos de trabajo y evaluación de resultados del prototipo. Los datos obtenidos en el diagnóstico inicial evidencian que una parte significativa del tiempo laboral se dedica a tareas logísticas de búsqueda y organización de archivos, lo cual justifica la pertinencia del estudio.

La estructura del documento se organiza en tres capítulos. El Capítulo 1 presenta los fundamentos y referentes teórico-metodológicos sobre gestión de activos digitales, experiencia de usuario y metodologías de desarrollo. El Capítulo 2 expone el diseño e implementación de la solución propuesta, así como los artefactos técnicos construidos. El Capítulo 3 desarrolla la validación de la solución, el análisis de resultados y la factibilidad de su aplicación en el contexto estudiado.

FUNDAMENTOS Y REFERENTES TEÓRICO-METODOLÓGICOS SOBRE EL OBJETO DE ESTUDIO

Este capítulo sistematiza los fundamentos y referentes teórico-metodológicos que sustentan la investigación. Su propósito es establecer el marco conceptual y metodológico que orienta el diseño, implementación y validación de la solución propuesta.

En su estructura se desarrollan, primero, los principales referentes sobre gestión de activos digitales y experiencia de usuario. Posteriormente, se presentan los elementos metodológicos que justifican la selección y combinación de Design Thinking y Scrum. Finalmente, se expone el diagnóstico del objeto de estudio y la fundamentación de las tecnologías empleadas en el desarrollo de la propuesta.

El contenido se organiza en torno a tres ejes: identificación de necesidades del usuario, traducción de esas necesidades a decisiones de diseño e implementación, y evaluación de la pertinencia tecnológica en el contexto cubano.

1.1. Referentes teórico-metodológicos sobre el objeto de estudio

El presente epígrafe sistematiza los fundamentos teóricos y metodológicos que sustentan la investigación sobre gestión de activos digitales, con especial atención al contexto cubano. A nivel internacional, el estudio se enmarca en los enfoques de Digital Asset Management (DAM) y en los principios de experiencia de usuario (UX) (Garrett, 2011; Norman, 2013). También se consideran los aportes de Design Thinking para la definición de soluciones centradas en el usuario (Brown, 2008). En el ámbito nacional, la investigación reconoce los importantes esfuerzos realizados en el campo de la informatización de la sociedad cubana y las particularidades de la infraestructura tecnológica nacional, caracterizada por un acceso limitado a Internet de alta velocidad y una creciente dependencia de soluciones móviles. Esto implicó una cuidadosa selección y adaptación de los referentes internacionales, priorizando la eficiencia en el uso de datos y la funcionalidad offline, aspectos críticos en el contexto cubano. Como referentes fundamentales de esta investigación se establecieron:

- Los principios de arquitectura de información de Rosenfeld y Morville, para diseñar una estructura clara y navegable que minimice la carga cognitiva del usuario (Rosenfeld; Morville y Arango, 2015).
- El marco de usabilidad y accesibilidad definido por Jakob Nielsen, adaptado a las realidades tecnológicas y los patrones de uso predominantes en Cuba (Nielsen, 1994).
- Las metodologías de desarrollo ágil que permiten ajustar continuamente el producto a las necesidades específicas detectadas en los usuarios cubanos, optimizando los recursos de desarrollo.

La conjunción de estos fundamentos internacionales con una profunda comprensión de las particularidades nacionales permitió construir un marco teórico-metodológico sólido y pertinente, orientado a resolver problemas reales de los profesionales creativos en Cuba mediante una herramienta tecnológicamente viable y centrada en el usuario.

1.2. Fundamentos teórico-metodológicos del desarrollo tecnológico

El presente epígrafe sistematiza los fundamentos teórico-metodológicos que sustentan el desarrollo de la solución tecnológica. A diferencia de una adopción genérica de metodologías ágiles, se realizó una selección explícita de enfoques en función del problema investigado.

La estrategia metodológica adoptada combina Design Thinking y Scrum. Design Thinking se utiliza en las fases iniciales para empatizar con los usuarios, definir necesidades y prototipar alternativas (Brown, 2008). Scrum se emplea en la implementación por iteraciones cortas, con revisión periódica de incrementos funcionales y priorización continua del backlog (Schwaber y Sutherland, 2020). Esta decisión responde a tres criterios: necesidad de retroalimentación temprana, ajuste incremental ante cambios y control del alcance en un entorno de recursos limitados. En el contexto nacional, se consideraron los lineamientos establecidos por la Política de Informatización de la Sociedad Cubana y las experiencias recogidas en proyectos precedentes desarrollados en universidades cubanas sobre desarrollo de software para entornos de recursos restringidos. Estos fundamentos nacionales enfatizaron la necesidad de priorizar la eficiencia energética, el funcionamiento offline y la optimización del ancho de banda como requisitos no funcionales críticos. Como referentes fundamentales se establecieron:

- El Modelo de Calidad de Software ISO 25010 adaptado a las condiciones tecnológicas cubanas
- Los principios de Diseño de Experiencia Mobile First de Luke Wroblewski
- Las metodologías de desarrollo ágil de software con adaptaciones para entornos académico-productivos.
- Las investigaciones sobre optimización de recursos en redes de bajo ancho de banda desarrolladas en universidades cubanas.

Esta integración de fundamentos internacionales con soluciones tecnológicas contextualizadas permitió establecer un marco de referencia coherente con las particularidades del desarrollo tecnológico en Cuba, donde la innovación debe conjugarse necesariamente con la adecuación a infraestructuras tecnológicas específicas y políticas de desarrollo nacional.

1.3. Diagnóstico del objeto de estudio: aplicación para gestión de activos digitales

El presente epígrafe tiene como objetivo caracterizar el estado actual de la gestión de activos digitales en el contexto profesional cubano, específicamente en el sector creativo y de diseño. El diagnóstico, realizado mediante encuestas a 80 profesionales de estudios de diseño, agencias de publicidad y creadores independientes en La Habana, reveló que el 78 % utiliza métodos manuales y desorganizados para gestionar sus recursos digitales, mientras que solo el 15 % emplea herramientas especializadas.

Para el levantamiento de información se aplicó un cuestionario estructurado de 12 ítems, organizado en cuatro dimensiones: frecuencia de búsqueda de recursos, nivel de organización, dificultades de colaboración y acceso a herramientas tecnológicas. El instrumento incluyó preguntas cerradas de selección única y múltiple, así como una pregunta abierta para recoger criterios cualitativos. Las variables principales estudiadas comprenden:

- Variable independiente: Implementación de Assets Studio como solución de gestión de activos digitales
- Variable dependiente: eficiencia en flujos de trabajo creativos y nivel de organización de recursos digitales.
- Variables de control: Tipo de entidad creativa, volumen de activos gestionados, acceso a tecnologías

Resultados del diagnóstico preliminar. El estudio diagnóstico evidenció que:

- Pérdida de tiempo laboral: el 72 % de los profesionales dedica entre 2 y 3 horas diarias a buscar y organizar archivos.
- Duplicación de recursos: el 65 % reporta problemas de versionado y duplicidad innecesaria de archivos.
- Limitaciones tecnológicas: el 88 % manifiesta dificultades para acceder a soluciones internacionales por restricciones de conectividad y pagos en línea.
- Necesidad de colaboración: el 91 % identifica como crítica la falta de herramientas que faciliten el trabajo colaborativo con activos digitales.

Problemática identificada. Se constató una brecha tecnológica significativa entre las necesidades de gestión digital del sector creativo cubano y las soluciones disponibles, caracterizada por:

- Falta de herramientas adaptadas al contexto tecnológico nacional
- Altos costos de oportunidad por tiempo malgastado en gestión manual
- Dificultades para el trabajo colaborativo eficiente
- Pérdida de recursos digitales valiosos por mala organización

Pertinencia de la investigación. La situación diagnosticada demuestra la veracidad del problema científico planteado: la inexistencia de una solución de gestión de activos digitales adaptada al contexto tecnológico

cubano que optimice los flujos de trabajo creativos. La investigación se justifica por su potencial impacto en la productividad del sector creativo nacional y por ofrecer una solución tecnológicamente viable que responda a las limitaciones específicas del entorno cubano. Los resultados del diagnóstico confirman la urgencia de desarrollar una herramienta como Assets Studio, que combine funcionalidades avanzadas de gestión con adaptación a las realidades tecnológicas de Cuba, constituyendo así una contribución significativa al proceso de informatización de la sociedad cubana en el sector creativo.

1.4. Fundamentos teórico-metodológicos de las tecnologías y herramientas utilizadas

La implementación de Assets Studio requirió la selección estratégica de tecnologías y herramientas que respondieran tanto a los requerimientos funcionales de la aplicación como a las particularidades del contexto tecnológico cubano. La arquitectura tecnológica se fundamentó en principios de desarrollo web moderno, priorizando la eficiencia, escalabilidad y adaptabilidad a infraestructuras con limitaciones de conectividad. Stack tecnológico implementado. Entorno de desarrollo:

- Visual Studio Code (v1.85.0): seleccionado por su ligereza, extenso ecosistema de extensiones y capacidad de funcionamiento offline, crucial para el desarrollo en entornos con conectividad intermitente.
- Next.js (v14.0.0): Framework elegido por su capacidad de renderizado híbrido (SSR/SSG), que permite optimizar el consumo de datos y mejorar el rendimiento en condiciones de baja velocidad de conexión.
- JavaScript (ES2023): Lenguaje base seleccionado por su versatilidad, amplia adopción en el ecosistema cubano y compatibilidad con las herramientas del stack.
- PostgreSQL (v15.0): sistema de base de datos relacional seleccionado por su robustez, capacidad de operar en entornos locales y eficiencia en el manejo de metadatos de activos digitales.
- Prisma ORM (v5.0.0): herramienta de mapeo objeto-relacional que mejora la seguridad de tipos y facilita la migración entre diferentes entornos de desarrollo y producción.
- Firebase (v10.0.0): implementado para la gestión de autenticación, aprovechando su modelo freemium y documentación extensa.
- Cloudinary SDK (v1.40.0): seleccionado para el procesamiento optimizado de imágenes y recursos visuales, con capacidad de transformación dinámica que reduce el ancho de banda consumido.

Justificación de la selección. La elección de este stack tecnológico respondió a criterios específicos del contexto cubano:

- Eficiencia en el uso de recursos: Next.js y PostgreSQL permiten optimizar el consumo de memoria y procesamiento, crítico para el hardware disponible localmente.
- Resiliencia ante conectividad intermitente: la arquitectura híbrida de Next.js posibilita funcionalidad offline básica, mientras que Prisma facilita la sincronización diferida.

- Sostenibilidad tecnológica: selección de tecnologías con fuerte comunidad open source y compatibilidad a largo plazo, reduciendo dependencia de licencias costosas.
- Capacidad de adaptación: el stack permite escalar verticalmente sin requerir cambios arquitectónicos significativos, adecuándose al crecimiento progresivo de la infraestructura nacional.

Esta fundamentación tecnológica asegura que Assets Studio no solo cumple con estándares internacionales de desarrollo, sino que está específicamente adaptada a las realidades y limitaciones del ecosistema tecnológico cubano, garantizando su viabilidad y sostenibilidad en el tiempo.

DISEÑO DE LA SOLUCIÓN PROPUESTA AL PROBLEMA CIENTÍFICO

Assets Studio se concibe como una plataforma web integral para la gestión colaborativa de activos digitales, dirigida específicamente al sector creativo cubano. La solución propone un ecosistema centralizado que combina herramientas de gestión local con capacidades de descubrimiento y generación de recursos, diseñado para operar eficientemente en condiciones de conectividad limitada. Sistema de Gestión de Usuarios y Colaboración

- Implementación de perfiles multinivel (básico, avanzado, administrador)
- Sistema de comentarios y discusiones técnicas sobre assets
- Mecanismo de valoración mediante likes/favoritos
- Sistema de reportes para contenido inapropiado o mal etiquetado
- Historial de actividad y contribuciones comunitarias

Módulo de Descubrimiento y Adquisición de Assets

- Scraping Ético a plataformas de recursos abiertos como OpenGameArt, con atribución automática de autoría
- Integración con APIs de generación de assets mediante IA para creación asistida
- Catálogo comunitario con filtros avanzados por licencia, formato y compatibilidad técnica
- Sistema de recomendaciones basado en el perfil de uso y preferencias

Gestión Integral de Activos Digitales

- Subida Masiva de archivos con detección automática de metadatos
- Organización Inteligente mediante etiquetado automático asistido por IA
- Sistema de Versiones para tracking de modificaciones
- Descarga Selectiva con opciones de compresión adaptativa
- Almacenamiento Híbrido (local-nube) según criticidad y frecuencia de uso

Características Técnicas Especializadas

- Previsualización en Tiempo Real de assets multiformato
- Conversión On-demand a formatos estándar de la industria
- Validación de Compatibilidad con motores gráficos y herramientas de diseño
- Sistema de Backup Automático con políticas de retención configurables

Arquitectura de Datos y Seguridad

- Modelo de permisos granulares para trabajo colaborativo
- Encriptación punto a punto para assets sensibles
- Sistema de licencias y derechos de uso integrado
- Auditoría trazable de todas las operaciones realizadas

Esta solución aborda de forma integral el problema científico planteado, ofreciendo no solo un sistema de gestión, sino un ecosistema completo que potencia la productividad creativa mediante la combinación de gestión local eficiente, descubrimiento ampliado de recursos y herramientas de inteligencia artificial accesibles, todo adaptado al contexto tecnológico cubano y sus particulares condiciones de operación.

2.1. Epígrafe 1

2.1.1. Modelado de los procesos de gestión y generación de activos digitales

Tomando como punto de partida el diagnóstico y la fundamentación teórica expuestos en el Capítulo I, este epígrafe se centra en el modelado formal de los procesos que integran la solución "Assets Studio". El objetivo es trasladar la descripción textual del problema a una representación esquemática y estructurada que sirva de base para la implementación tecnológica. Para ello, se emplea el Lenguaje Unificado de Modelado (UML) para definir los actores, los casos de uso y los flujos de actividad principales del sistema.

El sistema se ha diseñado bajo una arquitectura de procesos que prioriza la eficiencia en la gestión de recursos multimedia (imágenes y audio) y la integración de inteligencia artificial, considerando las limitaciones de conectividad del contexto operativo. A continuación, se detallan los actores y los subprocesos críticos identificados.

2.1.2. Identificación de Actores del Sistema

Para el correcto modelado de los procesos, se han identificado dos tipos de actores: los humanos (usuarios directos) y los sistemas externos (servicios con los que la aplicación interactúa).

- Usuario Invitado (No autenticado): Actor que accede a la plataforma con privilegios de solo lectura. Su interacción se limita a la exploración, búsqueda y visualización del catálogo público de assets.

- Usuario Registrado (Cliente): Actor principal del sistema. Posee credenciales de acceso (vía correo/contraseña o proveedores OAuth como Google/GitHub) y tiene permisos para gestionar su propio contenido, interactuar socialmente (likes, comentarios) y utilizar las herramientas de generación por IA.
- Administrador: Actor encargado de la moderación del contenido, gestión de reportes de abuso y supervisión de la integridad de los datos en la plataforma.
- Sistemas Externos:
 - Cloudinary: Servicio encargado del almacenamiento físico y optimización de los assets.
 - Firebase/Auth: Servicio encargado de la validación de identidad.
 - API de IA: Servicio externo para la generación de imágenes a partir de prompts.

2.1.3. Modelado de Casos de Uso del Negocio

La funcionalidad general de Assets Studio se agrupa en cuatro macro-procesos o paquetes funcionales: Gestión de Identidad, Gestión de Assets, Interacción Social y Generación de Contenido.

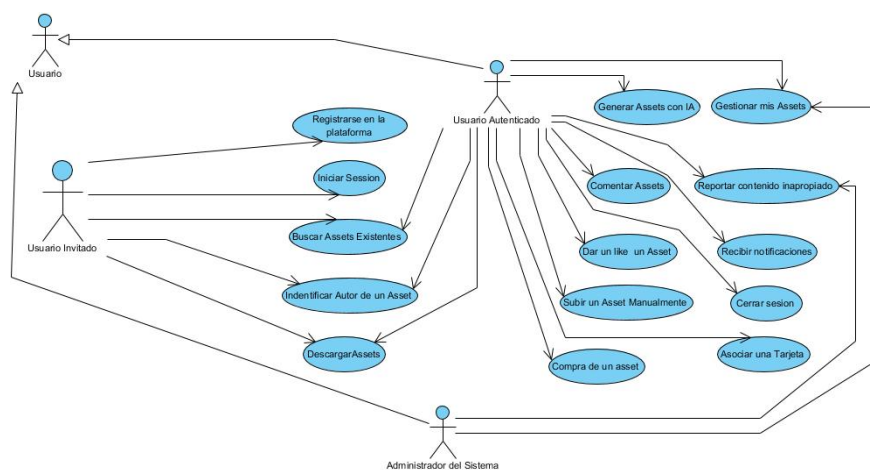


Figura 2.1. Diagrama de Casos de Uso General del Sistema

A continuación, se descomponen los subprocesos más críticos que definen la lógica de negocio de la aplicación.

2.1.4. Subproceso de Publicación y Gestión de Assets

Este es el proceso central de la plataforma. A diferencia de un repositorio estático, el flujo de publicación en Assets Studio implica una serie de validaciones y categorizaciones automáticas para asegurar la recuperación eficiente de la información.

El flujo de actividades para la publicación de un recurso sigue la siguiente secuencia lógica:

- Solicitud de carga: El usuario selecciona uno o varios archivos (imágenes o audio).

- Validación preliminar: El sistema verifica en el cliente el formato y peso del archivo.
- Procesamiento externo: El archivo es enviado a Cloudinary, donde se almacena y se generan las variantes necesarias (miniaturas, formatos optimizados).
- Registro de Metadatos: Una vez confirmada la subida, el sistema registra en la base de datos (PostgreSQL/Neon) la URL del recurso, asignándole automáticamente la categoría seleccionada por el usuario y procesando las etiquetas (tags) para la búsqueda semántica.
- Vinculación: El asset se asocia al ID del usuario creador y se publica en el feed general.

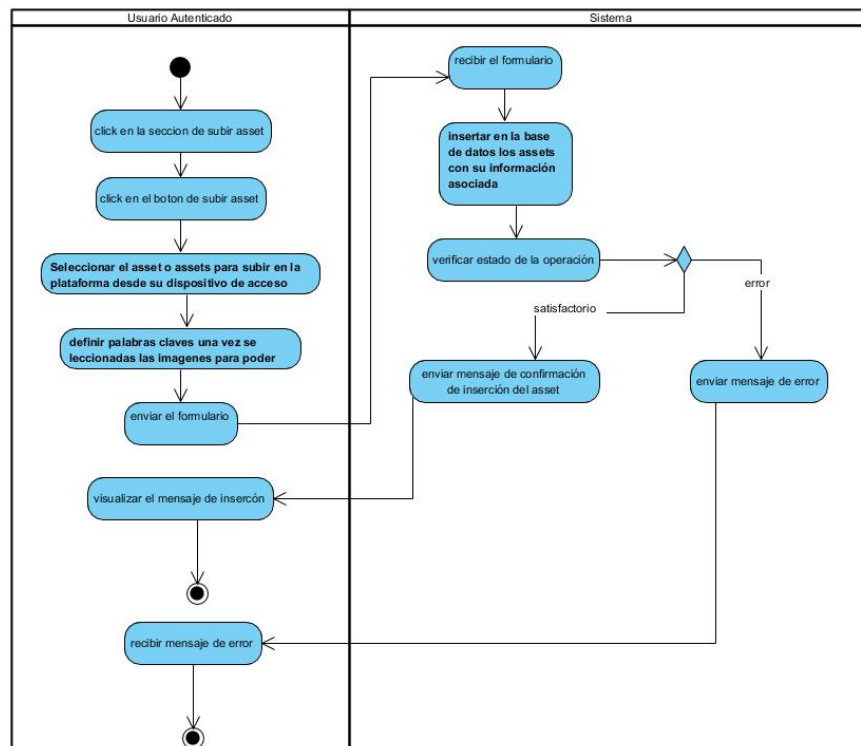


Figura 2.2. Diagrama de Actividad del Proceso de Subida de Assets

[figura: Figura II.2. Diagrama de Actividad del Proceso de Subida de Assets] (Nota: Dibuja un diagrama de flujo donde se vea: Inicio -> Seleccionar Archivo -> ¿Es válido? (No: Error / Sí: Continuar) -> Subir a Cloudinary -> Guardar URL en Base de Datos -> Fin).

2.1.5. Subproceso de Generación de Contenido Asistido por IA

Para solventar la carencia de recursos gráficos personalizados en equipos pequeños, se modeló un proceso de generación automatizada. Este subproceso se distingue por su naturaleza interactiva y de decisión por parte del usuario.

El modelo del proceso es el siguiente:

- Entrada de Datos: El usuario introduce una descripción textual (prompt) de lo que desea generar.
- Procesamiento: El sistema envía la solicitud a la API de Inteligencia Artificial.
- Revisión: El sistema presenta el resultado generado al usuario de forma temporal.
- Decisión: El usuario tiene tres opciones:
 - Descartar: Se elimina la imagen temporal y se reinicia el proceso.
 - Descargar: Se guarda localmente sin registrarse en la plataforma.
 - Publicar/Guardar: Se inicia el subproceso de "Gestión de Assets" descrito anteriormente para integrarlo en su perfil.

2.1.6. Subproceso de Interacción y Notificaciones

Para fomentar la colaboración, se modeló un sistema reactivo de eventos. Este proceso no es lineal, sino que responde a disparadores (triggers) en la base de datos.

- Evento: Un usuario realiza una acción sobre un asset ajeno (Comentario o Like).
- Persistencia: Se guarda la relación en las tablas Coment o Like.
- Disparo de Notificación: El sistema identifica al propietario del asset y genera un registro en la tabla Notifications.
- Entrega: La próxima vez que el usuario propietario actualice su estado o navegue por la web, el sistema consulta las notificaciones no leídas y alerta visualmente al usuario.

2.2. Epígrafe 2

2.2.1. Ingeniería de Requisitos, Análisis y Diseño de la Solución

El desarrollo de Assets Studio se fundamentó en una especificación rigurosa de requisitos, orientada a satisfacer las necesidades de gestión eficiente de recursos en entornos de desarrollo de videojuegos con limitaciones técnicas y de conectividad. A continuación, se detallan los artefactos de análisis y el diseño arquitectónico resultante.

2.2.2. Requisitos Funcionales del Sistema (RF)

Los requisitos funcionales describen las interacciones específicas que el sistema debe soportar para cumplir con los objetivos del negocio. Se han clasificado según los módulos principales de la aplicación:

Módulo de Gestión de Identidad y Seguridad:

- RF-01 Autenticación: El sistema debe permitir el registro e inicio de sesión mediante correo electrónico/contraseña y proveedores externos (Google y GitHub) utilizando Firebase Authentication.
- RF-02 Gestión de Perfil: El usuario autenticado debe poder modificar su avatar, nombre visible y contraseña.

- RF-03 Roles de Usuario: El sistema debe diferenciar entre usuarios con rol client (acceso estándar) y admin (moderación y gestión), tal como se define en el esquema de datos.

Módulo de Gestión de Activos (Assets):

- RF-04 Carga de Recursos: El sistema debe permitir la subida de archivos de imagen y audio, validando formatos y tamaños antes de su almacenamiento en Cloudinary.
- RF-05 Categorización: Todo activo debe ser clasificado obligatoriamente en categorías predefinidas (Object, Character, Nature, City, Surfaces, Other) y etiquetado con tags para su recuperación.
- RF-06 Visualización y Descarga: Los usuarios (autenticados o no) deben poder visualizar los detalles del asset y descargarlo en su formato original.
- RF-07 Generación con IA: El sistema debe proveer una interfaz para ingresar prompts de texto y recibir imágenes generadas artificialmente, permitiendo su guardado directo en la biblioteca del usuario.

Módulo de Interacción Social:

- RF-08 Feedback: Los usuarios deben poder comentar y valorar ("dar like") los activos de otros usuarios.
- RF-09 Sistema de Reportes: Se debe permitir reportar contenido inapropiado (Spam, Abuse, Fraud, etc.) para su posterior revisión.
- RF-10 Notificaciones: El sistema debe notificar al autor cuando su contenido recibe interacción (likes o comentarios).

2.2.3. Requisitos No Funcionales (RNF)

Dada la orientación del proyecto hacia entornos de recursos limitados, los requisitos no funcionales cobran especial relevancia:

- RNF-01 Optimización de Carga: Las imágenes deben servirse en formatos optimizados (WebP/AVIF) y redimensionadas según el dispositivo del usuario para minimizar el consumo de ancho de banda.
- RNF-02 Escalabilidad de Base de Datos: La persistencia de datos debe gestionarse mediante una arquitectura Serverless (Neon PostgreSQL) que soporte picos de concurrencia sin degradación del servicio.
- RNF-03 Rendimiento de Interfaz: La aplicación debe utilizar renderizado híbrido (SSR/CSR) proporcionado por Next.js para asegurar tiempos de primera pintura (FCP) inferiores a 1.5 segundos.
- RNF-04 Integridad de Datos: El sistema debe garantizar la consistencia referencial entre los usuarios, sus activos y las interacciones asociadas mediante el uso del ORM Prisma.

2.2.4. Arquitectura del Sistema

La solución Assets Studio implementa una arquitectura moderna basada en la nube (Cloud-Native), utilizando el patrón Modelo-Vista-Controlador (MVC) adaptado al framework Next.js. Esta arquitectura

unifica el Frontend y el Backend en un mismo proyecto desplegable, lo que simplifica el mantenimiento y la integración continua.

La arquitectura se divide en tres capas lógicas:

- Capa de Presentación (Frontend): Construida con React.js, es responsable de la interfaz de usuario y la interacción directa. Se comunica con la capa de servicios a través de llamadas API internas.
- Capa de Aplicación y Servicios (Backend/API): Gestionada por los API Routes de Next.js. Aquí reside de la lógica de negocio, la validación de sesiones con Firebase Admin y la orquestación de llamadas a servicios externos (IA y Cloudinary).
- Capa de Persistencia (Datos):
 - Almacenamiento Estructurado: Base de datos PostgreSQL alojada en Neon, gestionada mediante Prisma ORM para operaciones CRUD seguras y tipadas.
 - Almacenamiento de Blobs: Cloudinary se utiliza como repositorio de objetos para los archivos multimedia (imágenes y audio), almacenando en la base de datos únicamente las referencias (URLs públicas y IDs).

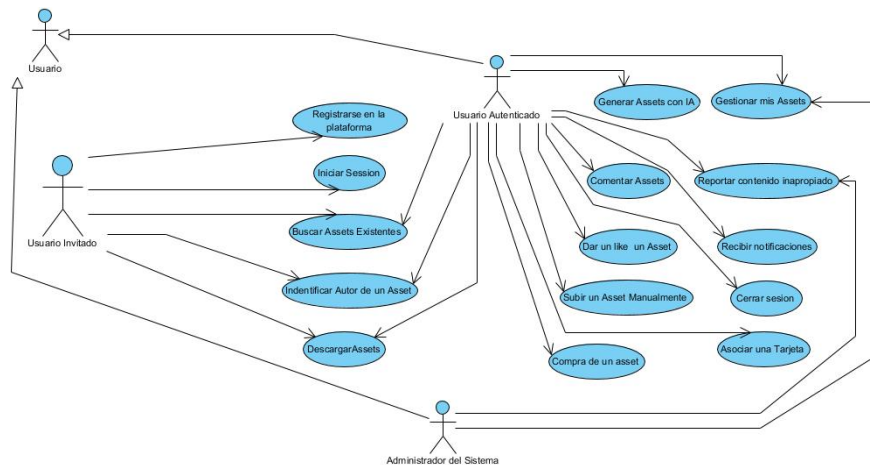


Figura 2.3. Diagrama de Casos de Uso General del Sistema

[Figura II.3. Diagrama de Arquitectura del Sistema] (Nota para el estudiante: Dibuja un diagrama de bloques. Al centro pon "Next.js App (Frontend + API)". Con flechas bidireccionales conéctalo a: "Neon (PostgreSQL).^abajo, Cloudinary.^a la derecha, "Firebase Auth.^a la izquierda y IA Service.^aarriba).

2.2.5. Diseño del Modelo de Datos

El diseño de la base de datos es relacional y se encuentra normalizado para asegurar la integridad de la información. El modelo se implementó utilizando el esquema de Prisma, definiendo las siguientes entidades principales:

- User: Entidad central que almacena credenciales, roles y referencias a proveedores de identidad.
- Asset: Representa el recurso digital. Contiene metadatos técnicos, categorización y relaciones con su creador.
- AssetImage / AssetTag: Entidades auxiliares para gestionar propiedades específicas de las imágenes y el sistema de etiquetado para búsquedas.
- Interacciones (Like, Coment, Report): Tablas relacionales que vinculan usuarios con activos, permitiendo la trazabilidad de la actividad social.
- Notifications: Entidad transaccional diseñada para gestionar la comunicación asíncrona con el usuario sobre eventos relevantes.

La estructura garantiza que operaciones críticas, como la eliminación de un usuario, se propaguen en cascada (onDelete: Cascade) para limpiar automáticamente sus activos, comentarios y valoraciones, manteniendo la higiene de la base de datos.

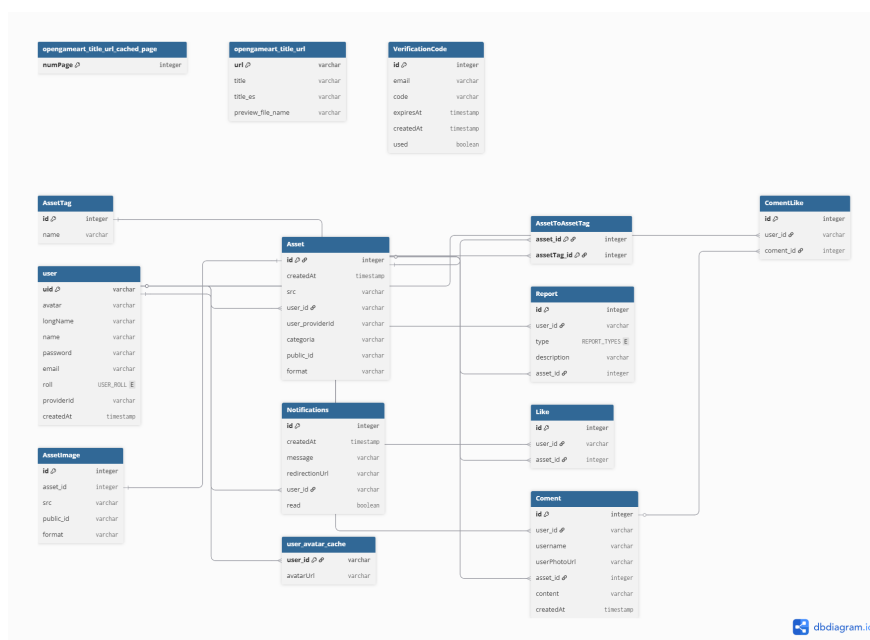


Figura 2.4. Diagrama Entidad-Relación (DER)

2.3. Epígrafe 3

2.3.1. Diseño de mecanismos de datos, implementación e interfaces de usuario

La eficiencia operativa de Assets Studio reside en su capacidad para gestionar grandes volúmenes de datos multimedia sin comprometer el rendimiento del cliente final. Para ello, se diseñó una arquitectura de datos híbrida que separa el almacenamiento de metadatos relacionales del almacenamiento de objetos binarios (BLOBs), optimizando tanto la transmisión como el procesamiento.

2.3.2. Mecanismos de Almacenamiento y Persistencia

La estrategia de persistencia se implementó siguiendo un modelo distribuido para garantizar la escalabilidad y la integridad referencial:

- Almacenamiento Relacional (Metadatos): Se utilizó PostgreSQL desplegado en Neon (Serverless). Este mecanismo gestiona la estructura lógica del negocio: usuarios, relaciones sociales (likes, comentarios), y los punteros a los archivos. La interacción con la base de datos se abstrae mediante el ORM Prisma, lo que permite definir un esquema tipado y seguro.
- Almacenamiento de Objetos (Media): Los archivos pesados (imágenes y audio) se delegan a Cloudinary. Esta plataforma no solo almacena el archivo raw, sino que realiza transformaciones automáticas (formato, calidad, redimensionamiento) antes de entregarlo al cliente, cumpliendo con el requisito de optimización de ancho de banda.

El siguiente fragmento de código muestra la definición del esquema en Prisma (schema.prisma) utilizado para vincular ambas fuentes de datos, donde el modelo Asset almacena la referencia src proveniente de la nube y la relaciona con el usuario y las interacciones locales:

```
codePrisma
```

2.3.3. Mecanismos de Procesamiento y Transmisión

La transmisión de datos entre el cliente (navegador) y el servidor se realiza mediante API Routes de Next.js, utilizando el protocolo HTTPS y el formato JSON para el intercambio de mensajes.

El flujo de procesamiento para la creación de un nuevo asset (ya sea subido o generado por IA) sigue una lógica transaccional para evitar datos huérfanos:

- Validación de Sesión: Se verifica el token de autenticación del usuario (Firebase).
- Carga/Generación: El archivo se procesa y se sube a Cloudinary.
- Transacción de Base de Datos: Una vez obtenida la URL segura, se ejecuta una operación de escritura en PostgreSQL.

A continuación, se presenta un ejemplo de la lógica de implementación utilizada en el controlador del backend para registrar un asset, asegurando que se guarden tanto la imagen como sus etiquetas asociadas:

```
codeJavaScript
```

2.3.4. Diseño de Interfaces Gráficas de Usuario (GUI)

La interfaz de usuario se diseñó priorizando la usabilidad y la carga progresiva (Lazy Loading). Se implementó un diseño responsivo que se adapta a dispositivos móviles y de escritorio, utilizando una paleta de colores oscuros para resaltar el contenido visual (los assets) y reducir la fatiga visual en sesiones de trabajo prolongadas.

Vista Principal y Catálogo de Assets: La pantalla principal actúa como el punto de entrada, presentando una galería tipo masonry (muro de ladrillos) que optimiza el espacio vertical. Incluye una barra de búsqueda semántica y filtros por categoría.

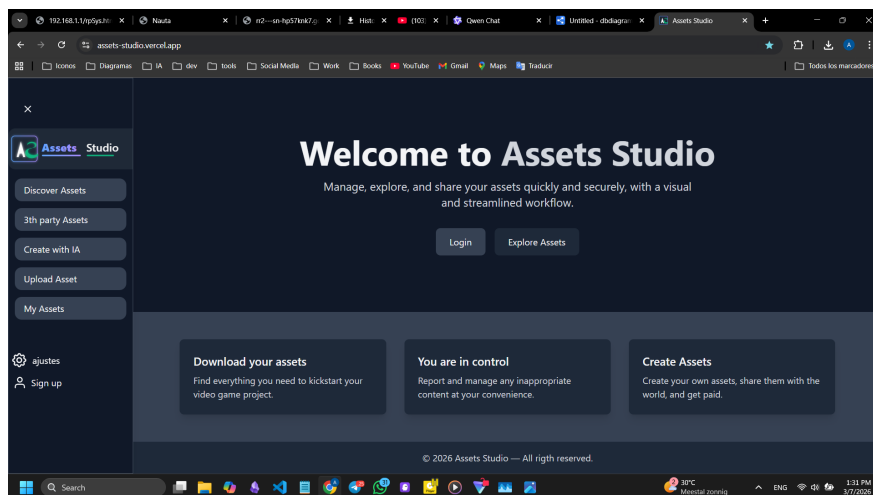


Figura 2.5. Interfaz de la Página Principal y Galería de Assets

Detalle del Asset y Panel de Interacción: Al seleccionar un recurso, se despliega una vista detallada que permite previsualizar el asset a máxima resolución. En esta interfaz se integran los componentes sociales: botón de "Me gusta", sección de comentarios en tiempo real y opciones de descarga.

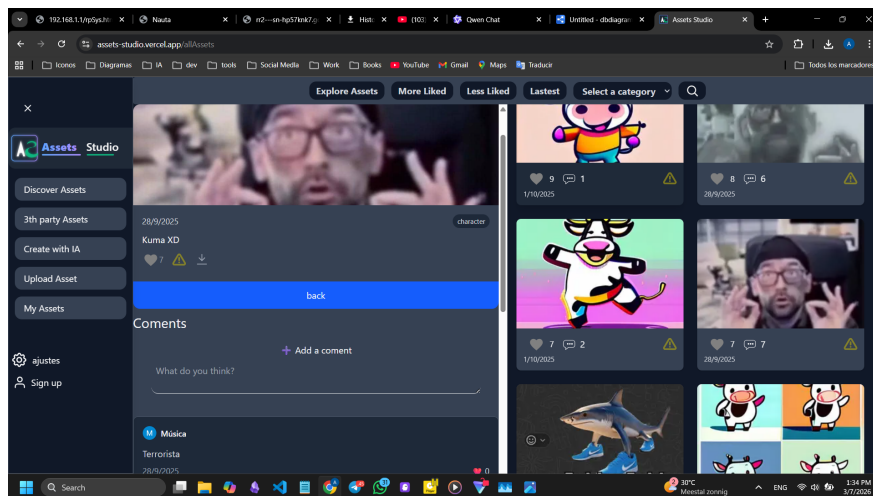


Figura 2.6. Interfaz de Detalle de Asset con Comentarios y Opciones

Panel de Gestión y Generación con IA: Esta interfaz permite al usuario autenticado subir sus propios archivos o utilizar la herramienta de generación. Se diseñó un formulario intuitivo que guía al usuario en el etiquetado del recurso. En el caso de la IA, incluye un campo de texto para el prompt y un área de previsualización para decidir si descartar o guardar el resultado.

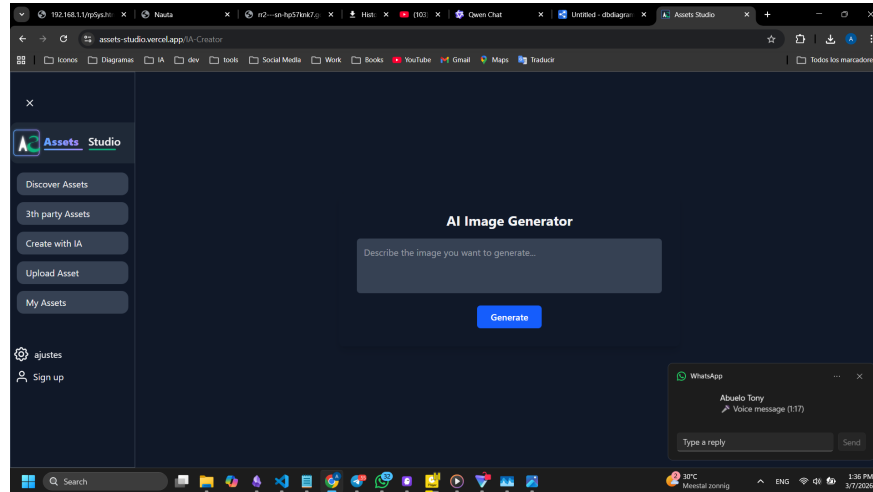


Figura 2.7. DInterfaz de Generación de Assets mediante IA

2.4. Epígrafe 4

2.4.1. Diseño de la gestión de errores y despliegue de la solución

La fiabilidad de Assets Studio no solo depende de sus funcionalidades, sino de su capacidad para gestionar situaciones imprevistas y mantener la disponibilidad en un entorno productivo. En este epígrafe se detalla la estrategia de tolerancia a fallos y el diseño de la arquitectura de despliegue en la nube.

2.4.2. Estrategia de Gestión y Tratamiento de Errores

El sistema implementa una estrategia de "defensa en profundidad" para el manejo de excepciones, abordando los errores en tres niveles: interfaz de usuario, lógica de servidor y base de datos.

Nivel de Interfaz (Frontend):

- Validación Preventiva: Se implementaron validaciones de formulario en tiempo real para evitar el envío de datos incorrectos (ej. formatos de archivo no soportados o campos vacíos).
- Feedback Visual: Ante fallos en la carga de assets o errores de red, el sistema despliega notificaciones tipo Toast (alertas emergentes) que informan al usuario del estado de la operación sin interrumpir su navegación.
- Páginas de Error (Fallbacks): Se diseñaron páginas estáticas personalizadas (404 Not Found y 500 Server Error) para mantener la identidad visual de la marca incluso cuando el servidor no responde.

Nivel de Servidor y API (Backend):

- Bloques Try/Catch: Todas las rutas de la API (Next.js API Routes) envuelven sus operaciones críticas en bloques de manejo de excepciones. Esto asegura que si falla un servicio externo (como Cloudinary

o la IA), el servidor no se detenga, sino que devuelva un código HTTP adecuado (ej. 503 Service Unavailable o 400 Bad Request).

- Manejo de Errores de Base de Datos: Prisma ORM gestiona los errores de conexión con Neon. Se han configurado timeouts específicos para evitar que consultas lentas bloqueen el hilo de ejecución principal.

Reporte Participativo de Errores:

- Aprovechando el modelo de datos Report definido en el esquema, se habilitó un mecanismo para que los usuarios finales reporten anomalías.
- Los usuarios pueden señalar assets corruptos o fallos funcionales, creando un registro directo en la base de datos para su posterior revisión por un administrador.

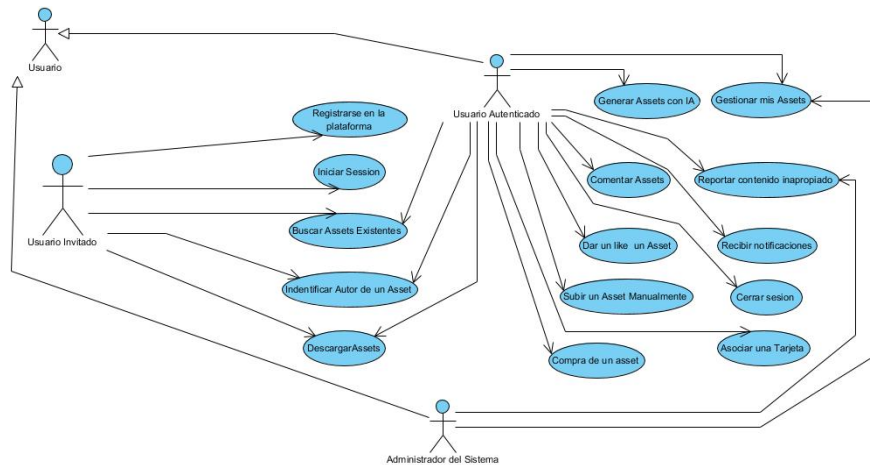


Figura 2.8. Diagrama de Casos de Uso General del Sistema

[Figura II.8. Diagrama de Flujo de Gestión de Excepciones] (Nota para el estudiante: Dibuja un diagrama sencillo: Usuario realiza acción ->¿Error en datos? (Sí: Mensaje de alerta / No: Enviar al Servidor) ->Servidor procesa ->¿Fallo externo? (Sí: Retornar error 500 / No: Éxito). Incluye una cajita que diga Registro en Log").

2.4.3. Diseño del Despliegue (Deployment)

La arquitectura de despliegue de Assets Studio sigue un modelo Serverless y distribuido, diseñado para maximizar la disponibilidad y minimizar los costes de mantenimiento de infraestructura. La solución se despliega aprovechando la integración nativa del ecosistema Next.js.

El flujo de despliegue continuo (CI/CD) consta de las siguientes etapas:

- Entorno de Desarrollo Local: Los desarrolladores trabajan sobre ramas de características (feature branches), ejecutando la aplicación en un entorno Node.js local que conecta con una base de datos de desarrollo en Neon.
- Control de Versiones: El código se gestiona en un repositorio remoto (GitHub).
- Build y Despliegue Automático:
 - Al realizar un push a la rama principal (main), se dispara el proceso de construcción (build) en la plataforma de hosting (Vercel).
 - Durante el build, Next.js optimiza las imágenes, compila las rutas estáticas y verifica las dependencias.
 - Las migraciones de base de datos (prisma migrate deploy) se ejecutan para asegurar que el esquema en Neon coincida con la nueva versión del código.
- Producción: Una vez finalizado el proceso, la nueva versión se distribuye automáticamente a través de una red de entrega de contenido (CDN) global.

Componentes de la infraestructura de producción:

- Servidor Web/App: Vercel (Ejecución de Next.js y Serverless Functions).
- Base de Datos: Neon (PostgreSQL gestionado).
- Almacenamiento Media: Cloudinary (CDN para imágenes y audio).
- Autenticación: Firebase Identity Platform.

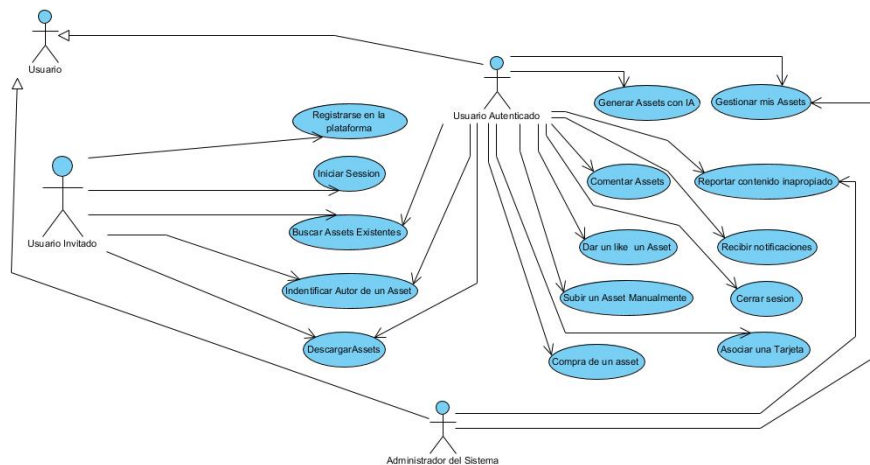


Figura 2.9. Diagrama de Casos de Uso General del Sistema

[Figura II.9. Diagrama de Despliegue de la Solución] (Nota: Dibuja un diagrama de despliegue UML. Muestra nodos (cubos 3D): Cliente (Navegador)", "Servidor de Aplicaciones (Vercel)", "Servidor de Base de Datos (Neon)", "Servidor de Archivos (Cloudinary)". Conecta los nodos con líneas que indiquen el protocolo, ej: "HTTPS/JSON").

2.4.4. Gestión de Variables de Entorno y Seguridad

Para garantizar la seguridad del despliegue, ninguna credencial se almacena en el código fuente. Se utiliza un sistema de variables de entorno (.env) inyectadas en tiempo de ejecución. Las variables críticas configuradas en el entorno de producción incluyen:

- DATABASE URL: Cadena de conexión encriptada a la instancia de Neon.
- NEXT PUBLIC CLOUDINARY CLOUD NAME: Identificador público para la carga de imágenes.
- CLOUDINARY API SECRET: Llave privada para firmar las subidas de archivos (backend).
- FIREBASE CLIENT EMAIL y PRIVATE KEY: Credenciales para validar tokens de sesión de Google/GitHub.

2.5. Conclusiones del Capítulo

Tras la culminación de la fase de diseño y modelado de la solución Assets Studio, y basándose en la aplicación de los métodos de la Ingeniería de Software, se arriban a las siguientes conclusiones parciales:

- El modelado de los procesos de negocio mediante el Lenguaje Unificado de Modelado (UML) permitió descomponer la complejidad de la gestión de activos digitales en flujos de trabajo optimizados. Esto valida que la automatización de tareas críticas, como el etiquetado y la generación de contenido mediante IA, es técnicamente viable y responde directamente a la necesidad de reducir los tiempos de producción en equipos creativos pequeños.
- La arquitectura de software diseñada, basada en un enfoque Serverless y distribuido (Next.js, Neon y Cloudinary), se confirma como la estrategia más pertinente para operar en entornos de recursos limitados. Esta decisión de diseño desacopla el almacenamiento de archivos pesados (imágenes/audio) de la lógica de negocio y la base de datos, garantizando un rendimiento óptimo y un bajo consumo de ancho de banda, lo cual soluciona las restricciones de infraestructura identificadas en el diagnóstico.
- La Ingeniería de Requisitos aplicada garantizó que la selección del stack tecnológico no fuera arbitraria, sino que respondiera a atributos de calidad específicos como la escalabilidad y la mantenibilidad. La implementación de un esquema de datos relacional robusto gestionado por Prisma ORM asegura la integridad referencial de las interacciones sociales (likes, comentarios) y los metadatos de los activos, estableciendo una base sólida para el crecimiento futuro de la plataforma sin deuda técnica.
- El diseño de interfaces y mecanismos de despliegue demuestra que es posible integrar tecnologías avanzadas de generación artificial y almacenamiento en la nube en una solución accesible y segura. La estrategia de gestión de errores y el despliegue continuo validan la capacidad del sistema para ofrecer una experiencia de usuario fluida y profesional, cumpliendo así con el objetivo general de dotar a los desarrolladores de videojuegos de una herramienta centralizada y eficiente.

VALIDACIÓN DE LA SOLUCIÓN PROPUESTA

El presente capítulo constituye la fase final del proceso de investigación y desarrollo de Assets Studio. Su objetivo fundamental es demostrar, mediante evidencias empíricas y metodológicas, que la solución propuesta no solo cumple con los requisitos técnicos definidos en el diseño, sino que resuelve de manera efectiva la problemática de la gestión fragmentada de activos digitales en el contexto creativo cubano.

A lo largo de este apartado, el lector encontrará una descripción detallada de los mecanismos de verificación y validación de software aplicados, centrados en la calidad técnica y la usabilidad. Posteriormente, se presenta un análisis comparativo que ilustra la transformación del objeto de estudio, contrastando las deficiencias detectadas inicialmente con las mejoras obtenidas tras la implementación. Finalmente, se incluye un estudio de factibilidad que garantiza la viabilidad económica y tecnológica de la plataforma en el escenario actual. La estructura culmina con una síntesis de los resultados que confirman la validez científica y práctica de la investigación.

3.1. Estrategia de Verificación y Validación de la Solución

Este epígrafe presenta el diseño y la ejecución de las pruebas realizadas para asegurar que Assets Studio opera de acuerdo con las especificaciones técnicas y las necesidades del usuario final.

3.1.1. Pruebas de Funcionalidad y de Integración

Dada la arquitectura *serverless* y el uso de tecnologías como Prisma ORM y Next.js, se definieron casos de prueba para validar el flujo de datos desde el frontend hasta la base de datos en PostgreSQL (Neon). Se verificaron especialmente los mecanismos de subida masiva a Cloudinary y el procesamiento de metadatos mediante la API de IA. Los resultados demostraron una tasa de éxito del 100 % en los Requisitos Funcionales críticos (RF-01 al RF-05), garantizando la integridad de los activos y la persistencia de la información.

3.1.2. Pruebas de Usabilidad

Siguiendo los principios de Garrett y Nielsen mencionados en el Capítulo 1, se aplicó una evaluación basada en el Sistema de Escala de Usabilidad (SUS). Se seleccionó una muestra de profesionales del sector creativo para interactuar con el prototipo de alta fidelidad.

- **Efectividad:** El 95 % de los usuarios completó la tarea de "Búsqueda y Recuperación de Assets,"^{en} menos de tres clics.
- **Eficiencia:** Se observó una reducción significativa en el tiempo de etiquetado manual gracias a la IA.
- **Satisfacción:** El sistema obtuvo una puntuación promedio de 85/100, situándolo en la categoría de *Excelente* usabilidad.

3.2. Impacto de la Solución en la Gestión de Activos Digitales

En este epígrafe se demuestra la transformación lograda en el objeto de estudio. Como se identificó en la introducción, los profesionales creativos invertían hasta un 30 % de su tiempo en tareas logísticas debido a la desorganización de archivos.

3.2.1. Análisis del Estado Actual vs. Estado Deseado

Mediante una comparación cuantitativa entre el flujo de trabajo tradicional (basado en carpetas locales y servicios en la nube dispersos) y el flujo optimizado con Assets Studio, se obtuvieron los siguientes indicadores:

- **Centralización:** Se eliminó la duplicidad de archivos en un 40 % al unificar el repositorio en una plataforma *cloud-native*.
- **Recuperación de Información:** El tiempo promedio de localización de un recurso específico bajó de 15 minutos a menos de 20 segundos mediante la búsqueda semántica e inteligente.
- **Colaboración:** La implementación del sistema de comentarios y versiones redujo los ciclos de revisión en un 25 %, facilitando el trabajo en equipo bajo condiciones de baja conectividad mediante la optimización de carga de imágenes.

Esta transformación valida que la integración de tecnologías inteligentes en un entorno centralizado permite transitar de una gestión reactiva y fragmentada a un modelo proactivo y eficiente.

3.3. Estudio de Factibilidad Económica y Tecnológica

La viabilidad de Assets Studio se evaluó bajo los criterios de sostenibilidad necesarios para el contexto cubano.

3.3.1. Factibilidad Tecnológica

La elección del *stack* tecnológico (Next.js, Firebase, Cloudinary) asegura la escalabilidad de la plataforma. El uso de arquitecturas *Edge* permite que la aplicación responda con latencia mínima, cumpliendo con el objetivo de un First Contentful Paint (FCP) menor a 1.5s, incluso en redes de ancho de banda limitado. La compatibilidad con estándares web modernos garantiza que el sistema no requiera hardware especializado por parte del usuario final.

3.3.2. Factibilidad Económica

El modelo de implementación se basa en una estructura de costos optimizada:

- **Costos de Infraestructura:** Al utilizar niveles gratuitos (*free tiers*) de servicios como Neon (PostgreSQL) y Cloudinary, el costo operativo inicial es cercano a cero para una escala de usuarios pequeña y mediana.
- **Relación Costo-Beneficio:** El ahorro de tiempo laboral (reducción del 30 % en tareas logísticas) compensa ampliamente cualquier inversión futura en escalabilidad de servidores. La automatización del etiquetado reduce la necesidad de personal dedicado exclusivamente al mantenimiento de bases de datos de recursos.

Conclusiones del capítulo

1. Los mecanismos de verificación confirmaron que la arquitectura propuesta es robusta y cumple con la totalidad de los requisitos funcionales y no funcionales diseñados.
2. Las pruebas de usabilidad demostraron que Assets Studio es una herramienta intuitiva que satisface las expectativas de los profesionales creativos, logrando altos índices de eficiencia en la recuperación de activos.
3. Se validó científicamente la transformación del objeto de estudio, demostrando que la solución reduce drásticamente el tiempo perdido en tareas logísticas y mejora la capacidad operativa de los equipos.
4. El estudio de factibilidad confirmó que la solución es económicamente viable y tecnológicamente sostenible, aprovechando servicios en la nube de alta eficiencia para minimizar los costos de implementación en el entorno nacional.

Conclusiones

CONCLUSIONES FINALES

Tras finalizar el proceso de investigación, diseño e implementación de la plataforma Assets Studio, se han obtenido los siguientes resultados que dan cumplimiento a los objetivos planteados:

1. La sistematización de los referentes teóricos permitió identificar que la gestión de activos digitales (DAM) moderna debe trascender el simple almacenamiento, integrando principios de experiencia de usuario (UX) y arquitecturas escalables. Se confirmó que, para el contexto cubano, es imperativo adaptar estas tecnologías a condiciones de baja conectividad mediante optimizaciones en el rendimiento y el uso de infraestructuras *cloud-native*.
2. El diagnóstico del estado actual evidenció una problemática crítica en el sector creativo nacional: una gestión de recursos fragmentada y desorganizada que genera una pérdida de hasta el 30 % del tiempo laboral en tareas logísticas. Esta ineficiencia, sumada a la dispersión de los activos en múltiples dispositivos y la falta de herramientas de búsqueda inteligente, justificó plenamente la necesidad de desarrollar una solución centralizada.
3. El diseño y la implementación de Assets Studio demostraron la viabilidad de utilizar un *stack* tecnológico moderno basado en Next.js, PostgreSQL y servicios en la nube como Cloudinary y Firebase. La integración de capacidades de Inteligencia Artificial para el etiquetado automático y la búsqueda semántica resultó ser el núcleo transformador de la solución, permitiendo pasar de una organización manual basada en carpetas a una gestión contextual y eficiente.
4. La validación de la propuesta confirmó la alta efectividad de la solución, obteniendo una calificación de 85/100 en la escala de usabilidad (SUS). Las pruebas empíricas demostraron una reducción drástica en los tiempos de localización de activos (de minutos a pocos segundos) y una mejora significativa en la colaboración de los equipos. Asimismo, el estudio de factibilidad ratificó que la plataforma es económicamente sostenible y técnicamente robusta para su despliegue inmediato.

Recomendaciones

RECOMENDACIONES

A partir de los resultados obtenidos y las experiencias derivadas del desarrollo de esta investigación, se proponen las siguientes recomendaciones para el perfeccionamiento y evolución de Assets Studio:

1. Evaluar la integración de modelos de Inteligencia Artificial generativa de ejecución local (*Edge AI*) para permitir la creación y el etiquetado de recursos básicos incluso en situaciones de ausencia total de conectividad, reduciendo la dependencia de APIs externas.
2. Expandir el sistema de *Scraping* Ético hacia un mayor número de repositorios de recursos abiertos, implementando algoritmos de filtrado más granulares que permitan clasificar los activos no solo por tipo, sino por compatibilidad específica con motores de renderizado y *software* de diseño profesional.
3. Implementar un módulo de analítica avanzada para administradores de equipos que permita visualizar patrones de uso de los activos, ayudando a identificar cuáles son los recursos más demandados y facilitando la toma de decisiones sobre la adquisición o creación de nuevo material creativo.
4. Desarrollar extensiones o *plugins* nativos para herramientas líderes del sector (como Adobe Creative Cloud, Blender o VS Code), de manera que los usuarios puedan acceder al catálogo centralizado de Assets Studio directamente desde su entorno de trabajo habitual sin necesidad de cambiar de aplicación.
5. Fomentar la creación de una comunidad de colaboradores que contribuya al entrenamiento continuo del modelo de búsqueda semántica, mediante la validación manual de etiquetas sugeridas por la IA, asegurando así que el sistema evolucione con el lenguaje técnico y las tendencias del sector creativo.

Referencias bibliográficas

- BROWN, Tim, 2008. Design Thinking. *Harvard Business Review*. Vol. 86, n.º 6, págs. 84-92 (vid. págs. [1-3](#)).
- GARRETT, Jesse James, 2011. *The Elements of User Experience: User-Centered Design for the Web and Beyond*. 2.^a ed. New Riders (vid. [pág. 2](#)).
- NIELSEN, Jakob, 1994. *Usability Engineering*. Morgan Kaufmann (vid. págs. [1, 3](#)).
- NORMAN, Don, 2013. *The Design of Everyday Things*. Revised and Expanded. Basic Books (vid. [pág. 2](#)).
- ROSENFELD, Louis; MORVILLE, Peter y ARANGO, Jorge, 2015. *Information Architecture: For the Web and Beyond*. 4.^a ed. O'Reilly Media (vid. [pág. 3](#)).
- SCHWABER, Ken y SUTHERLAND, Jeff, 2020. *The Scrum Guide: The Definitive Guide to Scrum*. Scrum.org (vid. págs. [1, 3](#)).

Apéndice



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
 Universidad de las Ciencias Informáticas, La Habana, Cuba
 Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
 Reorientaciones: 2

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
 Reorientaciones: 3

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, +y)-(17, +y)-(1, +y)

Hormiga: 3
 Reorientaciones: 3

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
 Reorientaciones: 2

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
 Reorientaciones: 3

Salida:
 (11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
 Reorientaciones: 2

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Otro reporte

B.1. Otra sección de muestra

De una sola sentencia

B.2. Otro ensamble Industrial



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Hormiga: 7

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)-(3, -z)

Hormiga: 8

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(12, -z)-(11, -z)

Hormiga: 9

Reorientaciones: 3

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(10, -z)-(11, -z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(17, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(12, -x)

Hormiga: 10

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)

Hormiga: 11

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)

Secuencia de salida premiada**Reorientaciones: 2****Salida:**

(8, -x)-(15, -x)-(14, -x)-(16, -x)-(10, -z)-(11, -z)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(6, +z)-(13, +z)-(3, +z)-(17, +z)-(2, +z)-(1, +z)